Comp 4911 Technical Review

Jakarta EE 10

Sample code updated to ee10:
https://github.com/bdlink/oreilly-ejb-6thedition-book-examples.git
use ee10 branch

Introduction

Chapter 1

Enterprise JavaBeans

- EJBs are components
  - That live on a server
- EJB is a standard
  - Allows distributed server components to be designed that plug together easily
  - Allows multiple vendors to supply compliant application servers
  - Similar in concept to JavaBeans
- Programmers don't have to deal with
  - Multithreading, transactions, connection pooling
  - All handled by the EJB server

EJB Goals

- Clearly define how application is partitioned
  - Client – EJBs – Servers
- Improve scalability and performance over ad-hoc technologies
- Increase reliability by encapsulating EJB
- Centralize resources for easier management
- Promote reuse
- Support different types of clients with one component technology

Component Types

Chapter 2

Three Kinds of EJBs

- Session Beans
  - Provide application services to client
  - Represent *business logic and services*
  - May be stateless, stateful (have client sessions) or singleton
- Message-Driven Beans
  - Respond to JMS messages – client ignorant of bean details
- Entity Beans (sort of)
  - Represent *business objects* in the application
  - Can allow container to make persistent or do it themselves (perhaps using JDBC)

Session Beans

- May implement `SessionBean` interface
- Represent big chunks of related *application functionality*
  - Perform *actions* on behalf of client
- Interacts with a *single client* at a time
- Can handle transactions
- Not intended to represent persistent data
  - Stateful ones should be serializable so bean can be written to file and restored
- Relatively short lived – from the client viewpoint

Message-Driven Beans

- May implement `MessageDrivenBean` interface
- Responds to JMS messages from client
  - Can be selective and only process messages on a specific Topic or Queue
- Reduces coupling between Client and bean
  - Client does not need to know about EJB model
- Server listens for messages to send to bean
  - Bean can be activated when there are messages to process

JPA Persistence Beans

- EJB 3.0 and 3.1 two types of entity beans
  - Legacy entity beans which *are* EJBs
    - Not used on new systems
  - Entity beans satisfying the Java Persistence Architecture (JPA) – *not* really EJBs
- JPA or Jakarta Persistence Beans are POJOs
  - Can be managed by container
    - Controlled by jakarta.persistence.EntityManager
  - If unmanaged, behave as any other Java Object

Model Taskflow: Session Beans

- Entity beans represent data, session beans represent process or actions
- Session beans are a good place to put business logic
- Session beans will use entity beans
  - Typically using a local interface for efficiency
- Clients will usually only call session or message-driven beans
  - Seldom entity beans directly
  - Unmanaged entity beans can be used as Data Transfer Objects (also called Value Objects)

Container Services

Chapter 3

Container Services

- Dependency injection
- Concurrency
- Instance pooling/caching
- Transactions
- Security
- Timers
- Naming and object stores
- Interoperability
- Lifecycle callbacks
- Interceptors
- Java Enterprise Platform integration
- Persistence

Jakarta Persistence

- Entities are Java classes with annotations
- Bean instances are attached or detached from container management
  - using EntityManager service
- EntityManager provides methods to create, find, query, update, and remove beans
- When detached, bean instances can be sent across the network
  - serving as data transfer objects

Object-to-relational Persistence

- Jakarta Persistence spec covers how to map object fields to a relational database
  - previously this was vendor specific
- For prototyping you can allow the server to do the mapping
- For operational systems, you probably want to do the mapping yourself
  - to control the database design

Simple ORM Example

```
@Entity
@Table(name="CABIN")
public class Cabin implements java.io.Serializable {
    private int id;
    private String name;
    private int deckLevel;

    @Id
    @Column(name="ID")
    public int getId() {return id;}
    public void setId(int pk) {id = pk;}

    @Column(name="NAME")
    public String getName() { return name;}
    public void setName(String str) {name = str;}

    @Column(name="DECK_LEVEL")
    public int getDeckLevel() {return deckLevel;}
    public void setDeckLevel(int level){deckLevel = level;}
}
```

What Does Persistence Do?

- You can map relationships and entities across multiple tables
  - 1-1, 1-n, n-1, n-n unidirectional or bidirectional relationships
  - Entity fields can come from a multi-table join
- Container keeps persistence bean instance consistent with the corresponding database entries
- No need for JDBC calls at all
- See chapters 9-14

Stateless Session Beans

Chapter 5

Three Session Beans

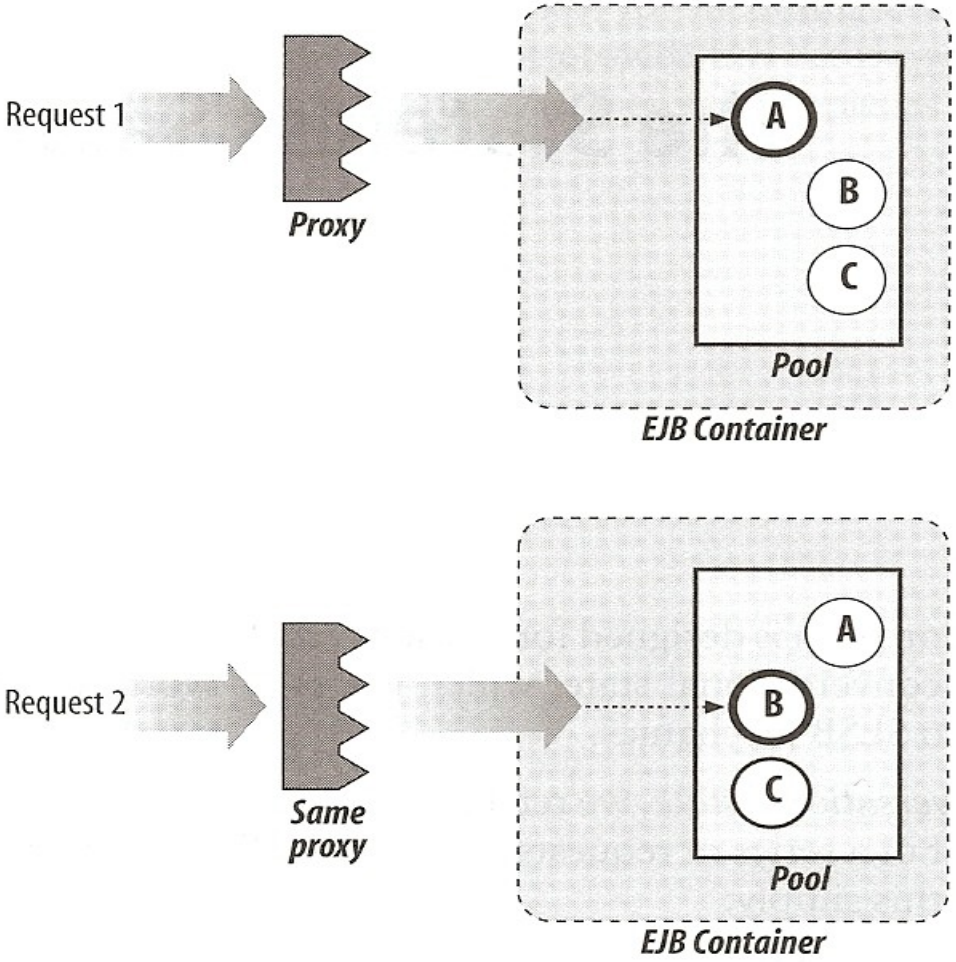
- Stateless, stateful and singleton
  - Short-lived, not persistent
- *Stateless* beans: collection of related services
  - Each service represented by a method
  - Each method invocation is independent of others
- *Stateful* beans hold client information between method calls
  - Represent logic that could have been stored in client application
  - Client holds bean instance which is not shared
  - Server may have timeout period to limit storage waste
- *Singleton* beans backed by single instance
  - All business requests serviced by same backing object

Stateless Session Bean (SLSB)

- Very efficient and easy to develop
- Bean instances shared between multiple clients
  - Because no conversational state maintained
- Does not require passivation or activation
- Each method should be self-contained
  - Completes an entire business task
  - Can run in a transaction

Session Bean Pooling

- The same proxy uses a different underlying bean instance
- Picked from the bean pool
- By the server

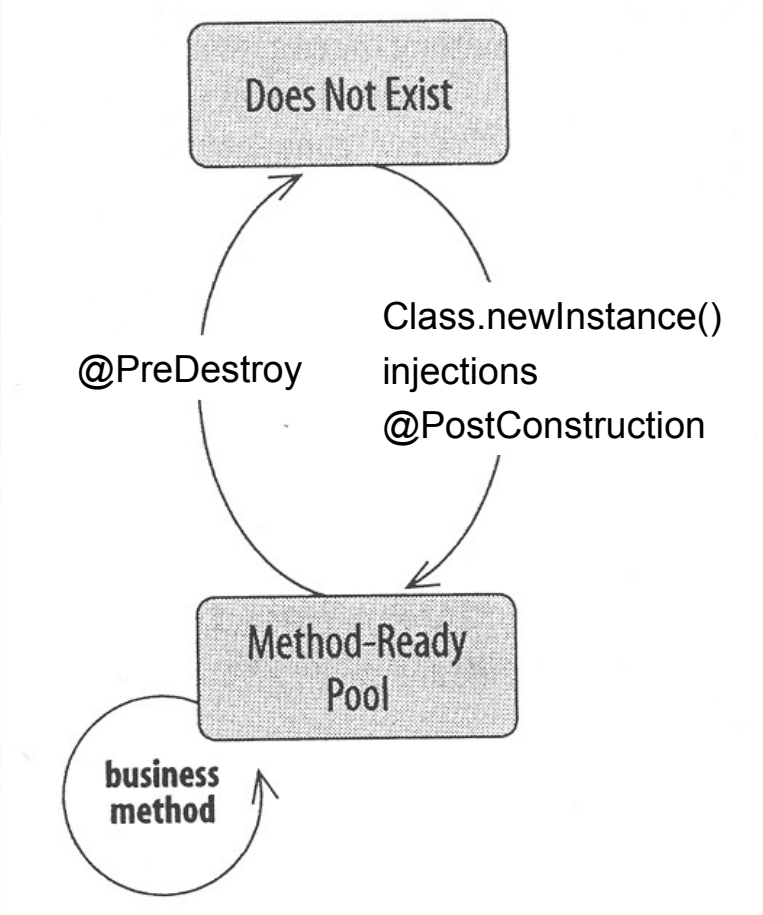


SLSB Restrictions

- Stores no client-specific information between method calls
- Can think of a method call as equivalent to a transaction
- Can store information from SessionContext and JNDI Enterprise Naming Context (ENC)
- Can have instance variables
  - Anything stored should be independent of client
  - No guarantee client will use same bean instance for successive method calls

Stateless Session Bean Life Cycle

- Only two states
  - Does not exist
  - method-ready pool
- Efficient to pool beans
  - ENC setup only required once



System Exceptions

- `java.lang.RuntimeException`
- Indicate serious programming errors
  - Unchecked, do not need to declare or catch
- Bean should not usually catch these
  - Let container handle them
  - Container will wrap in `EJBException`
- Bean can also throw `EJBException` for serious problems
- Jakarta Persistence uses runtime exceptions
  - Consider changing to application exceptions
  - Add `@jakarta.ejb.ApplicationException`

Application Exceptions

- EJB methods can throw application exceptions
  - Passed back to caller
- Indicate business logic error
  - Not programming error
  - Here, problem performing encryption: EncryptionException
- Consider appropriate exception type
  - EJBException for detected system problems
  - Application exceptions for business logic issues
  - Sometimes not obvious
    - EntityNotFoundException: system or logic error?
  - Transaction implications covered in Chapter 17

Exceptions from Other Subsystems

- Think how to handle these
  - such as `SQLException` from JDBC call
- Usually wrap in `EJBException` and rethrow
  - Otherwise container will pass them back to client
  - Interface will need to declare them
- Note `EJBException` is an unchecked exception
  - Avoids having to declare or catch these
- If you know the cause
  - Wrap as an appropriate application exception

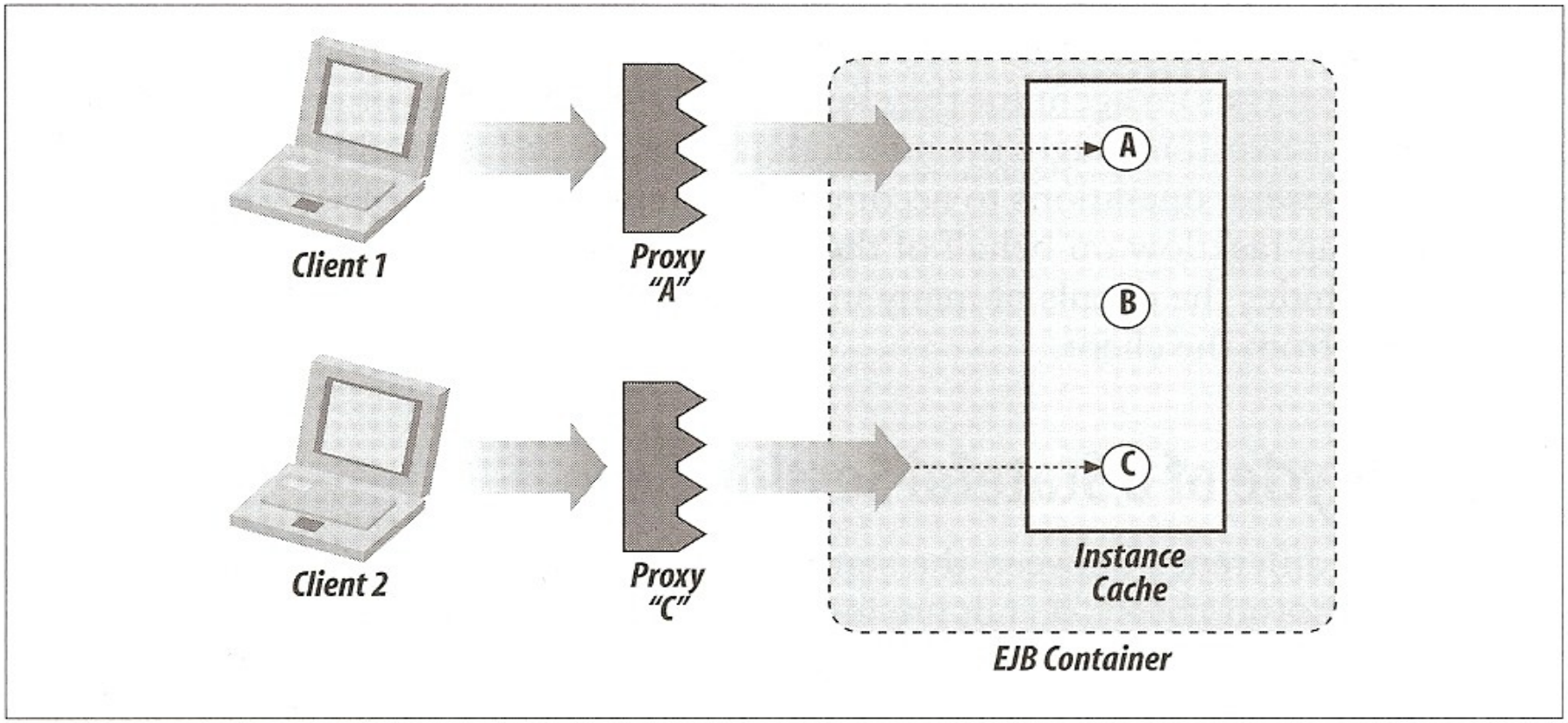
Stateful Session Beans

Chapter 6

The Stateful Session Bean

- Stateful session beans are dedicated to one client for the life of the bean instance
  - Paired with EJBObject (the proxy) for life
  - Unlike stateless ones
- Bean instance can keep conversational state about object
  - Methods can be interdependent
  - Methods can affect subsequent method calls
- This allows thinning the client and moving state information to the server
- Not for long-term storage, may time out
  - Not saved when server shuts down

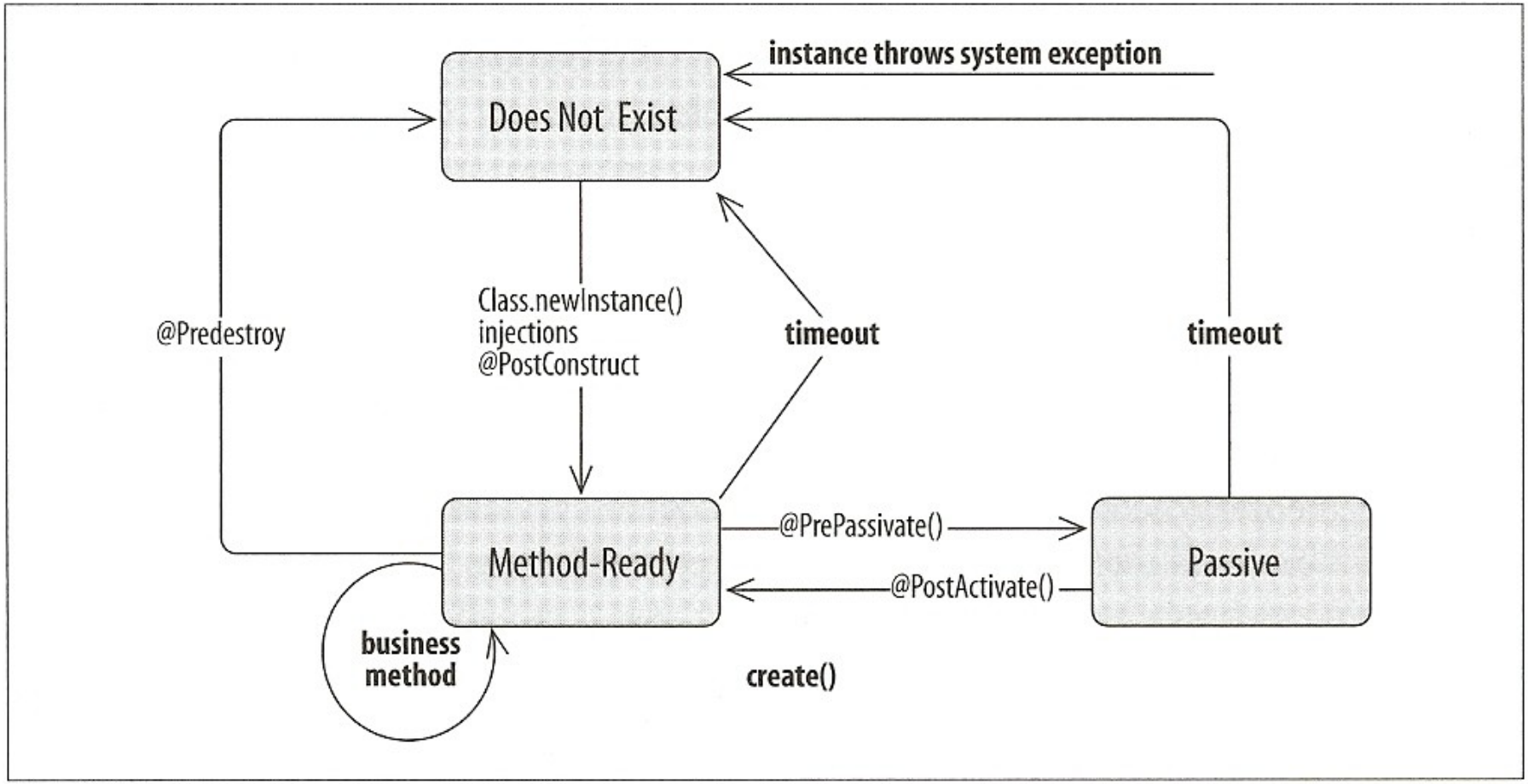
Client Relation with EJB Container and Bean



SFSB Life Cycle

- SFSBs do not use instance pooling
  - SLSBs do
- If bean is idle, container can remove it from memory and garbage collect instance
  - bean will be
    - *passivated* to preserve conversational state and
    - *activated* if it is used again
- Beans can implement `jakarta.ejb.SessionSynchronization`
  - Notify bean of its participation in transactions
    - Can cache DB data across method calls
  - Discussed in the Transactions chapter 17

Session Bean Life Cycle (Without SessionSynchronization)



Passivated State

- Passivation saves bean's conversational state
- Bean's conversational state consists of
  - Bean's primitive values
  - Bean's serializable objects
  - Following special types:
    - `jakarta.ejb.SessionContext`
    - `jakarta.transaction.UserTransaction`
    - `javax.naming.Context` (when uses JNDI ENC)
    - `jakarta.persistence.EntityManager`
    - `jakarta.persistence.EntityManagerFactory`
    - References to managed resource factories such as `javax.sql.DataSource`
    - References to other EJBs

Passivation

- When bean about to be passivated, @PrePassivate method called
- Bean should
  - Close open resources
  - Set nontransient nonserializable fields to null
  - (fields labeled `transient` not saved)
- Container saves fields on previous slide
  - If all fields fall into categories in previous slide, do not need @PrePassivate method
- Bean instance is serialized and saved

Activation

- Client calls method of passivated EJB
  - Container activates its instance
- Activation reverses the process of passivation
  - Bean deserialized
  - SessionContext reference, bean references, managed resource factories restored
  - Transient fields unpredictible
  - @PostActivate method should
    - Restore open resources
    - Initialize transient and nonserializable fields

Singleton Session Beans

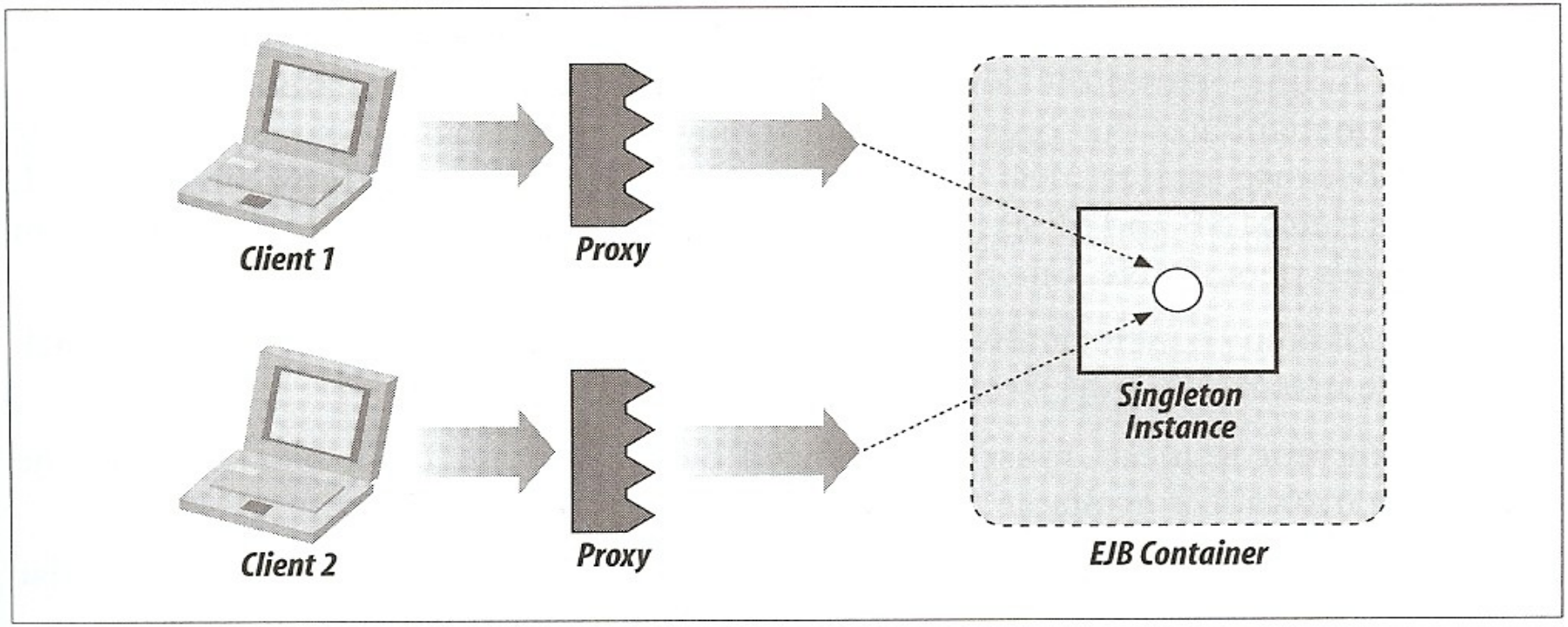
Chapter 7

Introduction

- Have seen the isolation that stateless and stateful session beans provide to clients
  - Each bean method is called by a single client thread at a time
  - Bean developer does not worry about concurrency
- Sometimes need to provide a single shared instance for all clients
  - EJB 3.1 adds singleton session bean
  - `@jakarta.ejb.Singleton`
  - Simple conceptual model

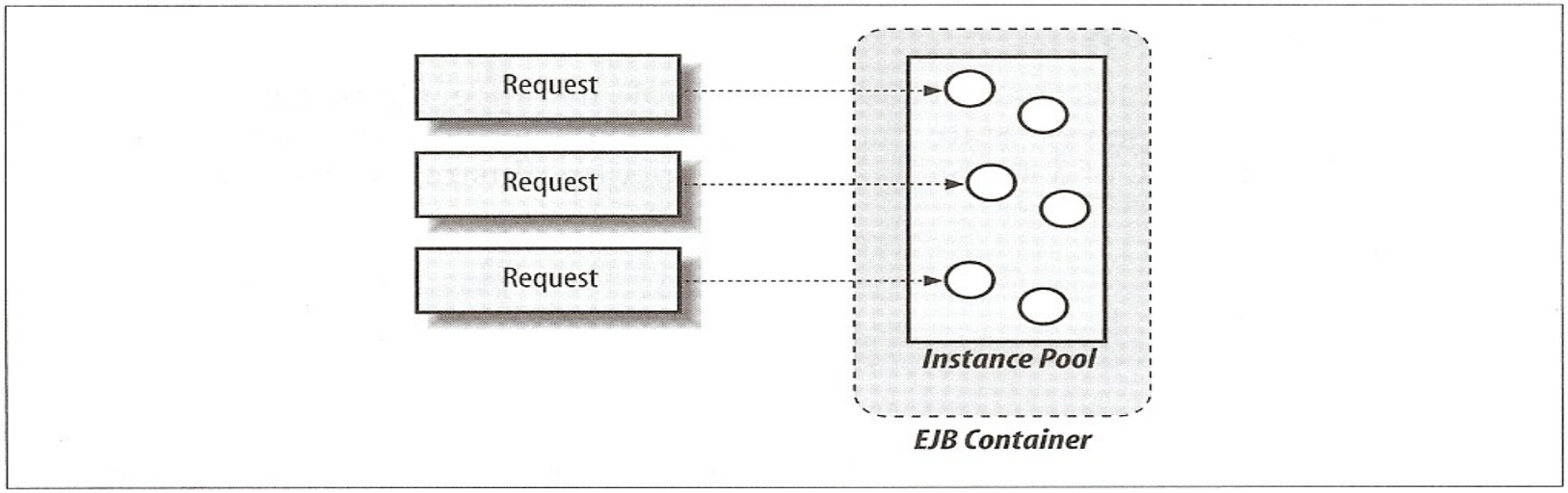
One Backing Instance

- All invocations on singleton session bean use same backing instance
  - Multiple calls at same time



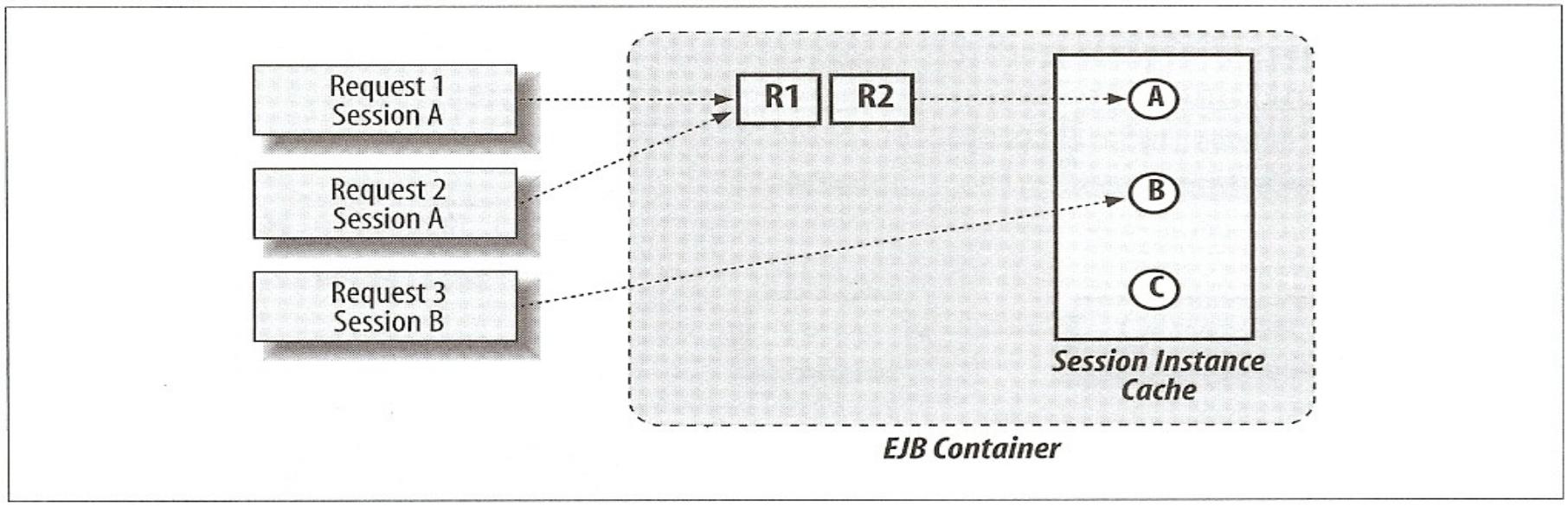
How is Concurrency Handled with EJBs?

- Have seen that for SLSB and SFSB only one request can access a bean instance at a time
  - Bean instance does not have to be thread-safe
- For stateless session beans each request goes to unique instance



Stateful Session Beans Queue

- Container serializes SFSB requests to same instance
  - Requests need to be queued by container
  - Infrequent occurrence



Container-Managed Concurrency

- Container will apply locking strategy
  - CMC is default for Singleton Session Beans
- Annotate class or methods with `@jakarta.ejb.Lock(`*`locktype`*`)`
  - locktype is enum defined in `jakarta.ejb.LockType`
  - READ allows concurrent access to all requests
    - As long as no write lock is currently held
  - WRITE (default) defines lock that can be held by one thread at a time
- Can specify timeout so calls do not block indefinitely
  - Use `@jakarta.ejb.AccessTimeout()`

Enterprise JavaBeans

Chapter 9

Persistence: EntityManager

Managed Versus Unmanaged

- When entity bean is *managed*
  - Also called *attached*
  - Manager tracks changes to the entity
  - Changes synchronized with database
    - When entity manager decides to flush its state
    - You have some control via `flush()`
- When entity bean is *unmanaged*
  - Also called *detached*
  - Any changes to entity are not tracked by the entity manager
  - Entity is independent from database

Persistence Context

- A set of managed entity object instances
  - Managed by an EntityManager
- When persistent context is closed
  - All of its managed entity instances become detached (unmanaged)
- Two types of persistence context
  - transaction-scoped and extended

Transaction-scoped Persistence Context

- Created in a JTA transaction and closed when the transaction completes
  - Jakarta Transactions API – See Ch. 17
- Restricted to container-managed persistence contexts
  - Injected with `@PersistenceContext` or XML equivalent
- Session bean methods are executed in a transaction by default
  - See Chapter 17 for lots more on this
  - When the method finishes, those beans become detached

Extended Persistence Context

- Allow entity beans to be managed outside of transaction boundaries
- Container-managed extended persistence contexts can be used in SFSBs
  - `@PersistenceContext(type=EXTENDED) EntityManager em;`
- Can be used by application code
  - (no requirement to be inside application server)

Example persistence.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
            https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
  version="3.0">
    <persistence-unit name="inventory-jpa">
        <jta-data-source>java:jboss/datasources/inventory
        </jta-data-source>
        <properties>
            <property name="hibernate.hbm2ddl.auto" value="validate"/>
            <property name="hibernate.show_sql" value="false"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

Container-Managed EntityManager

- Under Jakarta EE
  - Inject with `@PersistenceContext`
  - Uses JTA Transactions
  - Automatically closed when instance is discarded
    - you can not call its close method

```
@PersistenceContext(unitName="titan")
private EntityManager entityManager
```

- `type` can be TRANSACTION or EXTENDED
  - Either one for stateful session beans
  - Only TRANSACTION for stateless session and message-driven beans

Using an EntityManager

- `EntityManager` has methods to insert/remove/merge/query entities in a database
- `void persist(Object entity)`
- `void remove(Object entity)`
- `<T> T merge(T entity)`
- `void refresh(Object entity)`
- `<T> T getReference(Class <T> entityClass, Object primaryKey)`
- `boolean contains(Object entity)`
- `Query create…Query()` - query methods (Ch. 13)

Mapping Persistent Objects

Chapter 10

Employee Bean Class

```
package ca.bcit.infosys.domain;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Employee implements java.io.Serializable {
   private long id;
   private String name;

   @Id
   public long getId()  {return id;   }
   public void setId(long pk)  {id = pk; }

   public String getName() { return lastName; }
   public void setName(String lastName) { this.lastName = lastName; }

}
```

Default Mapping

- Unless otherwise specified, persistence provider assumes
  - DB Table name = Class name
  - DB Column name = property name
- Type will be assumed unchanged
  - Here long will map to database long
  - String will map to VARCHAR(255)
- @Id can be on a field or an getter
  - if on a field, the other fields are assumed to be persisted
  - If on an getter, the other JavaBean properties are assumed to be persisted
  - Mapping attributes are all on fields *or* all on properties

Elementary Schema Mappings

- First assume bean fields map to one table, but with different names or types
- `@jakarta.persistence.Table`
  - DB table name for the bean
- `@jakarta.persistence.Column`
  - Column properties for a bean property

Primary Keys

- Each entity bean must have a primary key
  - Must be unique
  - Can map to one or more properties
  - Must map to
    - Any Java primitive type or wrapper
    - `java.lang.String`
    - Primary key class composed of primitives or String fields

Single Property Keys

- These are the easiest to handle
- `@Id` annotates primary key field
- You can generate the primary key or have the persistence provider do it
  - Use `@GeneratedValue` annotation
    - `GenerationType strategy() default AUTO`
    - `String generator() default ""`
  - `enum GenerationType {TABLE, SEQUENCE, IDENTITY, AUTO, UUID}`

Composite Keys

- Sometimes necessary to have a composite primary key
- Two ways of mapping this
  - `@IdClass`
    - References external primary key class
    - Corresponding properties are stored in entity bean and labeled with `@Id`
  - `@EmbeddedId`
    - primary key object is a field/property of entity

Primary Key Class

- A JavaBean with the fields corresponding to the primary key
  - E.g. String lastName; long sin
- Three requirements
  - must be serializable
  - must have public no-argument constructor
  - must implement `equals()` and `hashCode()` methods
- See `ExternalEmployeePK.java`

Other Property Mappings

- `@Transient`
- `@Basic` and `FetchType`
- `@Temporal`
- `@Lob`
- `@Enumerated`
- See `EmployeeWithProperties.java`

`@Embedded` Objects

- You can embed non-entity Java objects as properties in an entity bean
  - Similar to embedding primary key objects
  - With no mapping, persistence provider will map these as `Blob`s if serializable
- Annotate the class with `@Embeddable`
- Annotate the property in the entity with `@Embedded`
  - You can map the properties of the embedded class using `@Column`
  - Optionally override the mapping with `@AttributeOverrides` in the entity
- See example in text

Entity Relationships

Chapter 11

Seven Relationship Types

- One-to-one, unidirectional
  - employee --> address
- One-to-one, bidirectional
  - employee <--> computer
- One-to-many, unidirectional
  - employee --> phone number
- Many-to-one, unidirectional
  - customer --> employee contact

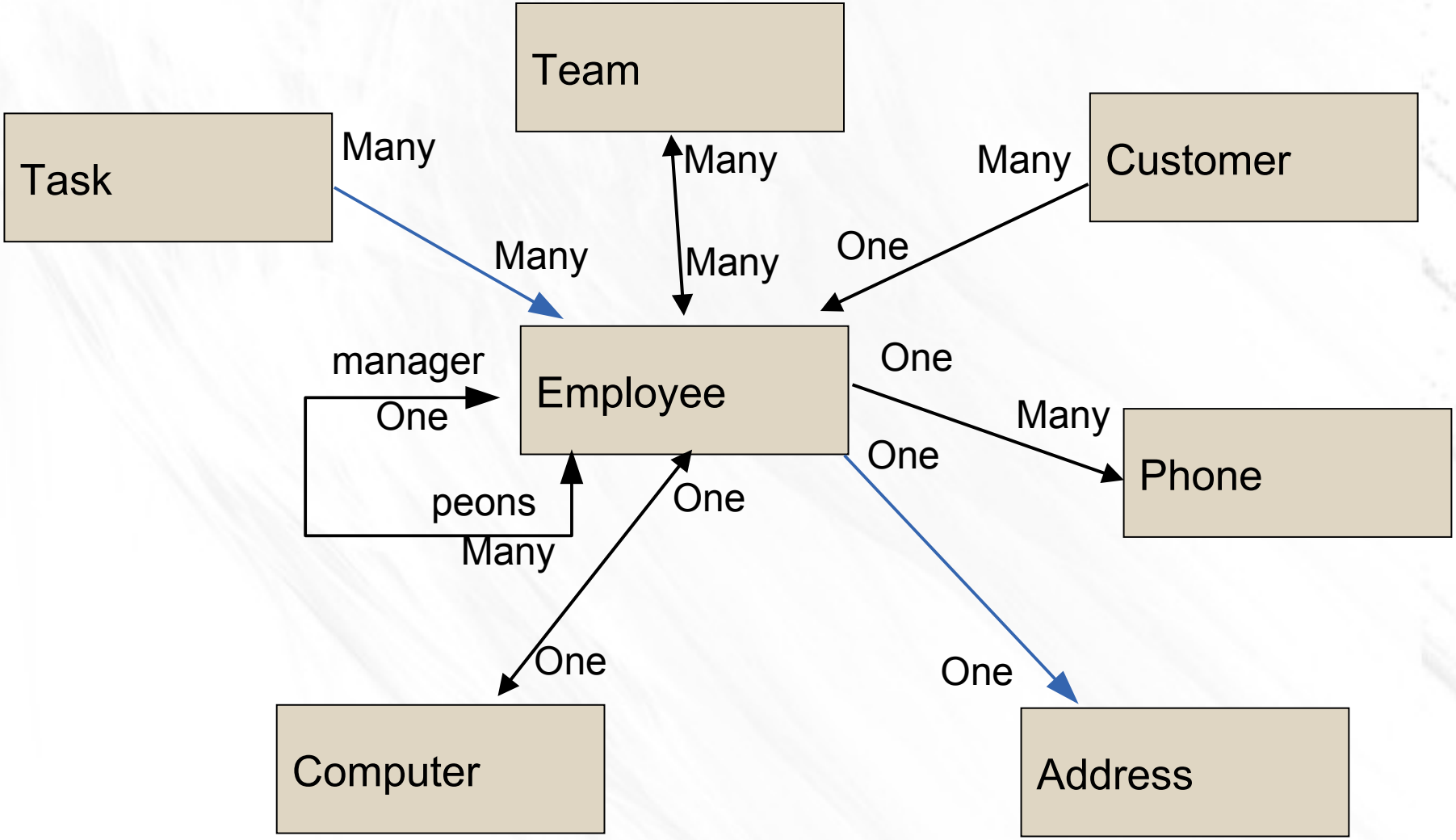
Seven Relationships

- One-to-many, bidirectional
  - employee (manager) <--> employee
  - many-to-one bidirectional is the same relationship as one-to-many bidirectional
- Many-to-many, unidirectional
  - task --> employee
- Many-to-many, bidirectional
  - employee <--> team

Navigation

- Navigation is an OO domain concept
  - Relational databases do not implement navigation
  - Databases use joins instead
- Example: employee --> address
  - Indicates can navigate from `Employee` to `Address`
  - Means `Employee` has address property
    - `myEmployee.getAddress()` returns the employee's address
- Example is 1-1
  - Later will see how to handle “many” case

Class Diagram



1. One-to-One, Unidirectional Relationship

- An Employee has one address and can find it
  - employee --> address
  - See example code
- `Employee` class has
  - `@OneToOne`
  - `private Address address;`
  - `getAddress()` and `setAddress()` methods
  - Address has no reference to Employee
- Address entity cannot be shared between two different Employee entities (why? 1-1)
  - It *is* allowed to have two relationship fields in same Employee entity to point to same `Address` entity
  - For example, add `workAddress` field

Database Modeling

- In our abstract persistence model the employee has a link to the address
  - `@JoinColumn` specifies foreign key column in employee table
- Databases can implement the relationship differently
  - Address with a foreign key for the employee
  - Associative table having keys to both the employee and address
  - Address and employee with the same primary key
- Entity mapping allows you to specify the database schema separately from the abstract persistence schema

Default Relationship Mapping: One-to-One Unidirectional

- If persistence provider provides auto schema generation
  - no need for `@JoinColumn` or `@PrimaryKeyJoinColumn`
  - Only need `@OneToOne` to specify multiplicity
- Persistence provider will generate foreign key mapping in source table
  - Employee in this example
  - Column name is
    - Property you are mapping + "_" + primary key column name of the referenced table
  - E.g. ADDRESS_ID in EMPLOYEE table

2. One-to-One, Bidirectional Relationship

- Example: Computer with one-to-one bidirectional relationship to Employee
  - So can find owner of computer
  - Database can use (same as 1-1 uni):
    - foreign key in both employee and computer tables
    - single foreign key in one of them
    - associative table
    - common primary key

Abstract Programming Model

- `Employee` (inverse side) has
  - `@OneToOne(mappedBy="owner")`
  - `private Computer computer;`
  - `public Computer getComputer();`
  - `public void setComputer (Computer computer);`
- `Computer` (owning side) has
  - `@OneToOne`
  - `private Employee owner;`
  - `public Employee getOwner();`
  - `public void setOwner (Employee owner);`

Note with Bidirectional Relationships

- One entity is *owning* side of relationship
  - here owner is Computer
  - Inverse side uses `mappedBy` attribute
- To change object related-to in database need to use setter on *owning* side
  - Should update both objects relationship fields
- E.g. to associate `aComputer` with different employee:
  - oldEmployee.setComputer(null)
  - newEmployee.setComputer(aComputer)
  - aComputer.setEmployee (newEmployee)

Default Relationship Mapping: One-to-One Bidirectional

- If persistence provider auto-generates schema
  - Need `@OneToOne` on owning side
  - Need `@OneToOne(mappedBy=...)` on inverse side
- Persistence provider generates foreign key column on owning side
  - *Property you are mapping + "_" + primary key column name of the referenced table*
    - Referenced table is the inverse side
  - here OWNER_ID in COMPUTER table

3. One-to-Many, Unidirectional Relationship

- Now we assume an employee may have multiple phone entities
  - Employee can find his/her phones but the phones cannot find the employee
  - The phones property will be some type of collection
- Database often represents this relationship with employee foreign key in the phone record
  - Backward pointer on the "many" side
  - Could also use associative join table as in many-many relationship

Abstract Programming Model

- `Employee` defines accessor methods for relationship as earlier
  - `@OneToMany(cascade={CascadeType.ALL})`
  - `@JoinColumn(name="EMPLOYEE_ID")`
  - `private Collection<Phone> phones;`
  - `public Collection<Phone> getPhoneNumbers(){...};`
  - `public void setPhoneNumbers (Collection<Phone> phones){...};`
- Phone does not have any methods for this relationship
- To add a phone number inside Employee:
  - `Collection<Phone> phoneNumbers = getPhoneNumbers();`
  - `phoneNumbers.add(phone);`
  - Since employee is owning side, phone will be added to DB

Default Relationship Mapping: One-to-Many Unidirectional

- Specify `@OneToMany` annotation on field/property of owning side
- Without `@JoinColumn` or `@JoinTable` default is to use a join table
  - Table name is *ownerTable* + "_" + *inverseTable*
  - Foreign keys
    - *owningEntity* + "_" + *owningEntityPK*
    - *relationshipProperty* + "_" + *inverseEntityPK*
- Here use EMPLOYEE_PHONE table
  - EMPLOYEE_ID FK column
  - PHONES_ID FK column (unique)

4. Many-to-One, Unidirectional Relationship

- Many `Customer`'s have one `Employee`
  - As contact
  - Can get from a `Customer` to the `Employee` (but not the other way)
- We have seen this pattern before
  - Add a `get/setEmployee` relationship method to the `Customer` entity:
  - `@ManyToOne`
  - `@JoinColumn(name="CONTACT_ID")`
  - `public Employee getPrimaryContact() {return primaryContact;}`
  - `public void setPrimaryContact(Employee primaryContact) {`
    `this.primaryContact = primaryContact;}`

Default Relationship Mapping: Many-to-One Unidirectional

- Only need `@ManyToOne` annotation
- Default is to use foreign key mapping
  - Owning table has foreign key for inverse table
  - Foreign key name is
    - *relationPropertyName* + "_" + *inverseTablePK*
  - in this case `PRIMARYCONTACT_ID` column in `CUSTOMER` table

5. One-to-Many, Bidirectional Relationship

- Same as Many-to-One bidirectional
- Employee (manager) and Employee (peons) have a one-to-many bidirectional relationship
- Employee entity has `get/setManager` relationship property that takes `Employee manager`
  - Annotated with `@ManyToOne`
  - Optional: `@JoinColumn(name="MANAGER_ID")`
- Employee entity has `get/setPeons` relationship property that takes `Collection<Employee>` reservations
  - Annotated with `@OneToMany(mappedBy="manager")`

Abstract Programming Model

- Owning side for one-to-many bidirectional is *always* the many-to-one side
  - Here: Employee (as peon)
  - This is side whose property is single valued and has foreign key (if using FK mapping)
- This means must call Employee.setManager whenever add or remove a Employee's manager
  - Required to update relationship in database
  - If always make sure to update both sides of the relationship you will be safe

6. Many-to-Many, Bidirectional Relationship

- Team and Employee have a Many-to-Many bidirectional relationship
- Database schema generally uses an association table
- If business rule wants each Employee to be included only once on the same Team
  - Team's set/getMembers can use Set<Employee> for the type, not a Collection
  - The Employee's set/getTeams uses Collection<Team> for its type as before

Abstract Programming Model

- Owning side (Team) specifies `@ManyToMany`
  - Inverse side `@ManyToMany(mappedBy="…")`
- Owning side may define `@JoinTable` to specifically map the association table
- To modify the relationship, use the owning side for persistence in the database
  - best to modify both sides consistently

Default Relationship Mapping: Many-to-Many Bidirectional

- Annotate field/properties with `@ManyToMany` and `@ManyToMany(mappedBy="`*`propertyName`*`")`
- Default mapping creates association table
  - Table name is *ownerTable* + "_" + *inverseTable*
  - Foreign keys
    - *relationshipPropertyInInverse* + "_" + inverse*EntityPK*
    - *relationshipPropertyInOwner* + "_" + owner*EntityPK*
- Here use TEAM_EMPLOYEE table
  - TEAMS_ID FK column
  - MEMBERS_ID FK column

7. Many-to-Many, Unidirectional Relationship

- `Employee` and `Task` have a many-many relationship
  - it is possible to find the employees associated with a task but not the other way around
- So in the Task entity define `get/setOwners` relationship methods
  - Use `Collection<Employee> owners`
- The Employee entities do not have a relationship field for tasks

Example

- In Task (nothing in Employee class)

```
@ManyToMany
@JoinTable(name="EMPLOYEE_TASK",
              joinColumns={@JoinColumn(name="TASK_ID")},
              inverseJoinColumns={@JoinColumn(name=
                                          "EMPLOYEE_ID")})
Collection<Employee> owners;
...
public Collection<Employee> getOwners() { return owners; }
public void setOwners(Collection<Employee> owners) {
               this.owners = owners;
}
```

Default Relationship Mapping: Many-to-Many Unidirectional

- Similar to Many-to-Many Bidirectional
- Annotate field/properties with `@ManyToMany`
- Default mapping creates association table
  - Table name is *ownerTable* + "_" + *inverseTable*
  - Foreign keys
    - *ownerEntity* + "_" + owner*EntityPK*
    - *relationshipProperty* + "_" + *inverseEntityPK*
- Here use TASK_EMPLOYEE table
  - TEAM_ID FK column
  - MEMBERS_ID FK column

Detached Entities and Lazy Loading

- Recall that we can set a field to be eagerly/lazily loaded
  - Lazy loading may not load the field until it is used
- If the entity is detached, the persistence manager is not present to load the field
  - So it may not be initialized if lazy loaded
- Implementation-dependent what happens if reference uninitialized field of detached entity
  - Hibernate throws LIE as we have seen

Entity Inheritance

Chapter 12

Single Table per Class Hierarchy

- With this strategy, one table has all of the fields for all classes in the hierarchy
  - Need a discriminator column to indicate which specific class is stored in a given row
  - Discriminator column type can be STRING (default), CHAR, or INTEGER
  - If discriminator column type is STRING, value defaults to simple class name
- See chapter 12 example code

`@Inheritance` Annotation

- This specifies which strategy to use
- One attribute
  - `InheritanceType strategy() default SINGLE_TABLE`
- Where `InheritanceType` is an enumeration with values
  - `SINGLE_TABLE, JOINED, TABLE_PER_CLASS`

`@DiscriminatorColumn`

- You can specify the name and type of the discriminator column with this
- Attributes:
  - `String name() default "DTYPE"`
  - `DiscriminatorType discriminatorType() default STRING`
    - other possible values `CHAR` or `INTEGER`
  - `String columnDefinition() default ""`
  - `int length() default 10`

Advantages/Disadvantages

- Advantages
  - Simple to implement and best performance
  - No joins, unions, or subselects required
- Disadvantages
  - All columns of subclass properties must be nullable
    - table is not normalized
    - takes more storage than needed
  - adding new subclass requires modifying table

Table per Concrete Class

- One table for each concrete class
  - with columns for all of its properties, including properties in its superclasses
  - Not required by current specification so non-portable
- No discriminator column needed in database schema
- Specify `@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)` for the base class of the hierarchy
- See chapter 12 example code

Advantages/Disadvantages

- Advantages
  - can put constraints, such as NOT NULL on the subclass properties
- Disadvantages
  - Database not normalized
    - redundant columns for base class's properties
  - Tricky to implement polymorphic relationships
    - inefficient if SQL UNIONS not used
  - Probably not your first choice to use

Table per Subclass

- Separate table for each subclass
  - with the properties defined in the subclass
- Called `JOINED` strategy
- *Some* persistence providers require a `DiscriminatorColumn` (e.g. Toplink)
- Persistence manager needs to do a join on the tables in the hierarchy to get all of the fields
  - `@PrimaryKeyJoinColumn` specifies join keys

Advantages/Disadvantages

- Advantages
  - Can define NOT NULL constraints on any column
  - Database is normalized
  - Faster than TABLE_PER_CLASS
  - Adding subclass does not require modifying existing tables
- Disadvantages
  - Slower than SINGLE_TABLE

Queries, Criteria API, and JPA QL

Chapter 13

Query API

- `jakarta.persistence.Query` is an interface
  - see next slide for methods
- Get a `Query` object from entity manager:
  - `Query createQuery(String qlString);`
  - `<T> TypedQuery<T> createQuery(CriteriaQuery<T> criteriaQuery);`
  - `<T> TypedQuery<T> createQuery(String qlString, Class<T> resultClass);`
  - `Query createNamedQuery(String name);`
  - `<T> TypedQuery<T> createNamedQuery(String name, Class<T> resultClass);`
  - `Query createNativeQuery(String sqlString);`
  - `Query createNativeQuery(String sqlString, Class resultClass);`
  - `Query createNativeQuery(String sqlString, String resultSetMapping);`

Some `Query` Methods

- `List getResultList()`
- `Object getSingleResult()`
- `int executeUpdate()`
- `Query setMaxResults(int maxResult)`
- `Query setFirstResult(int startPosition)`
- `Query setHint(String hintName, Object value)`
- `Query setFlushMode(FlushModeType flushMode)`
- `Query setParameter(String name, Object value)`
- `Query setParameter(String name, Date value, TemporalType temporalType)`
- `Query setParameter(String name, Calendar value, TemporalType temporalType)`
- Plus `setParameter` methods with position instead of name

Example

- Using `setParameter` to fill in named parameter

```
public List findByName(String name){
   Query query = entityManager.createQuery(
      "select e from Employee e
                        where e.name=:eName");
   query.setParameter("eName", name);
   return query.getResultList();
}
```

Constructor Expressions

- Can use constructor in a SELECT clause
  - To create a nonentity java object
  - Using specified columns for the constructor parameters
- E.g. suppose have defined Name
  - With a 2 argument constructor `Name(Long id, String name)`
- Can call constructor in query:
  - `SELECT NEW org.mypackage.Name(e.id, e.name) FROM Employee e`
  - Query object will allocate new Name for each row returned

`IN` Operator

- Allows identifier to represent individual elements in a collection-based relationship
  - Eg. return all phones of all employees
    - `SELECT p FROM Employee AS e, IN(e.phones) AS p`
- With identifier to represent individual elements of collection can use in further expressions
  - Eg. return all phone numbers for which employees have in some phone
    - `SELECT p.number FROM Employee AS e, IN(e.phones) AS p`
- Only includes Employees that have some phones

INNER JOIN

- Alternative syntax for `IN` which is similar to SQL
- `SELECT p.number FROM Employee e INNER JOIN e.phones p`
  - Same as `SELECT p.number FROM Employee e, IN(e.phones) p`
- Reserved word `INNER` is optional
  - E.g find phone numbers for all employees who have a task
    - `SELECT p.number FROM Task t JOIN t.owners o JOIN o.phones p`
    - `SELECT p.number FROM Task t, IN(t.owners) o, IN(o.phones) p`

Fetch Joins

- `JOIN FETCH` syntax allows preloading an entity's relationships even if they have `FetchType.LAZY`
- Reduces data base queries required
- E.g. suppose phones are lazy fetched
  - `SELECT e FROM Employee e LEFT JOIN FETCH e.phones`
    - Gets all employees with/without phone
  - Employees returned would have phone field prefetched
  - Uses one query instead of N+1 queries

Aggregate Functions in `SELECT` Clause

- `COUNT(`identifier or path expression`)`
- `MAX(`path expression`)`, `MIN(`path expression`)`
- `AVG(`numeric path expression`)`, `SUM(`numeric path expression`)`
- `DISTINCT` used with aggregate functions eliminates duplicate values
  - `SELECT DISTINCT COUNT(e.address.zip) FROM Employee e WHERE e.address.zip LIKE '554%'`
- Fields with *null* value eliminated from result set operated on by aggregate functions
- `COUNT` returns 0 when evaluating empty collection
- `AVG`, `MIN`, `MAX`, `SUM` return *null* when evaluating empty collection

`GROUP BY` and `HAVING`

- `GROUP BY` allows grouping data for aggregate functions
  - same as in SQL
  - must specify column from the `SELECT` clause
- `HAVING` restricts groups to ones satisfying a boolean condition
  - E.g. Names of employees that manage more than 10 peons with a count of the number they manage:
  - `SELECT e.name, COUNT(p) FROM Employee e JOIN e.peons p GROUP BY e.name HAVING COUNT(p) > 10`

Subqueries

- Subqueries are `SELECT` statements embedded in another query
- JPA SQL supports subqueries in `WHERE` and `HAVING` clauses
  - e.g. `SELECT COUNT(res) FROM Reservation res WHERE res.amountPaid > (SELECT AVG(r.amountPaid) FROM Reservation r)`
- Subquery can reference identifiers in `FROM` clause of outer query
  - e.g. `SELECT cr FROM Cruise cr WHERE 100000 < (SELECT SUM(res.amountPaid) FROM cr.reservations res)`

Subquery Modifiers: `ALL, ANY, SOME, EXISTS`

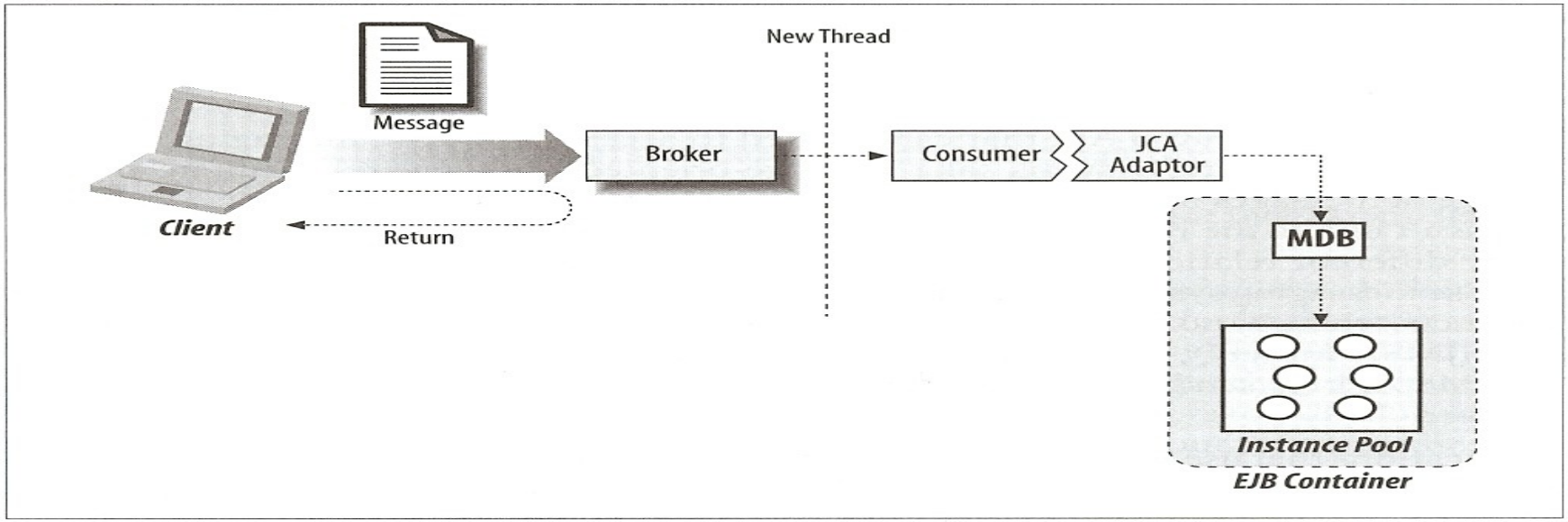
- These are quantifiers in front of the subquery
- `ALL` returns true if *all* the things in the subquery match the conditional expression
  - e.g. `FROM Cruise cr WHERE 0 < ALL (SELECT res.amountPaid FROM cr.reservations res)`
- `SOME` (or `ANY`) returns true if *anything* in the subquery matches the conditional expression
- `EXISTS` returns true if the subquery returns one or more values
  - e.g. `SELECT cr FROM Cruise cr WHERE EXISTS (SELECT res FROM cr.reservations WHERE res.amountPaid = 0)`

Message-Driven Beans

Chapter 8

Overview

- Message-driven beans are asynchronous listeners
- Clients send request to messaging system and go about their business



JMS Terminology

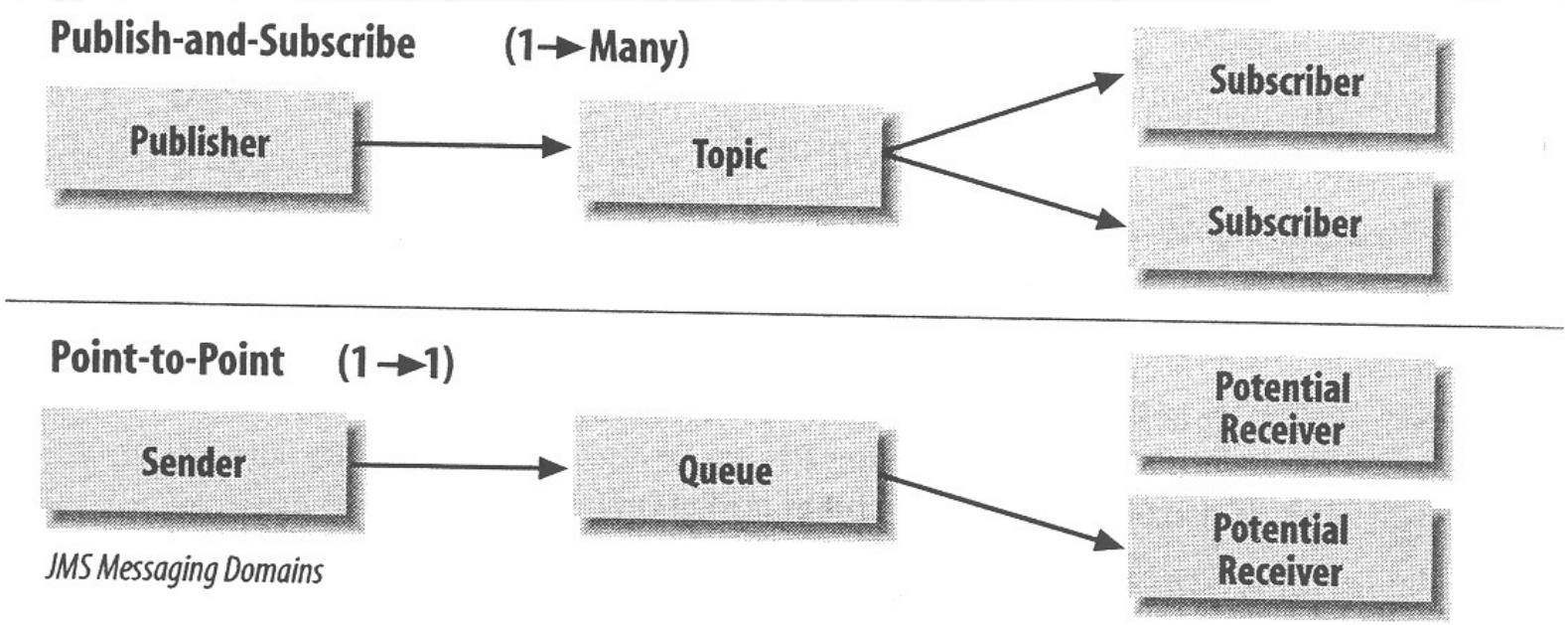
- Applications using JMS are *JMS clients*
- Messaging system is called a *JMS provider*
- *JMS Application* is business system with one or more JMS clients and a provider
- JMS client that sends a message is called a *producer*
- JMS client that receives messages is *consumer*
- All EJBs can be producers
- Java applications or message-driven beans can be consumers

JMS is Asynchronous

- Primary advantage of JMS messaging
  - Client can send a message without waiting for a reply
- Clients send message to JMS destination
  - Topic or Queue
- Consumer of messages will get message from JMS provider
- More convenient than RMI as means for distributing system across multiple platforms

JMS Messaging Models

- Two types, each with interfaces/classes
  - Publish-and-subscribe
  - Point-to-point



JMS Messaging Domains

JMS-Based Message-Driven Beans

- MDBs are
  - Stateless
  - Server-side
  - Transaction-aware components
  - For processing messages delivered via JMS
- Container manages
  - Transactions
  - Security
  - Resources
  - Concurrency and
  - Message acknowledgement
- Container can create multiple instances to handle high message traffic

@MessageDriven

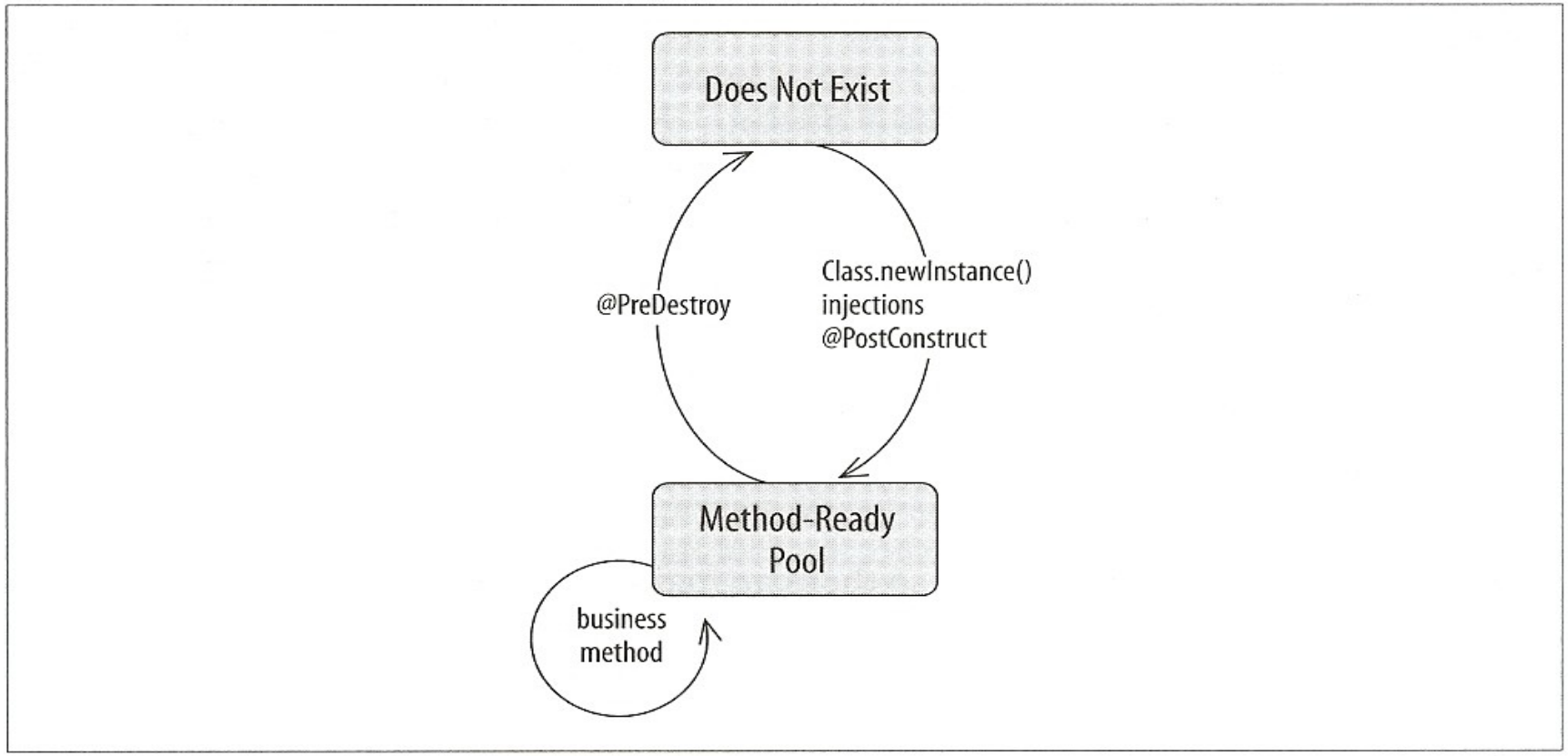
- MDBs identified with @MessageDriven annotation
  - or <message-driven> element of XML deployment descriptor
- @MessageDriven takes one attribute: activationConfig
  - which takes array of @ActivationConfigProperty annotations
  - Each @ActivationConfigProperty has two attributes:
    - propertyName and
    - propertyValue
- @ActivationConfigProperty describes messaging properties of MDB
  - For JMS includes: acknowledgeMode, messageSelector, destinationType, and subscriptionDurability

MDB `onMessage()` Method

- This is where all business logic goes
- Container passes messages to MDB by calling `onMessage(Message message)`
- `onMessage` is called by one thread at a time
- onMessage body parses message to know how to handle it
- Can use session or other beans as needed

Life Cycle of a Message-Driven Bean

- Similar to stateless session beans



Message Types

- Messages have headers and message bodies
  - Header has delivery information and metadata
- Message body has application data
- Several message types
  - `TextMessage, MapMessage, ObjectMessage, StreamMessage, BytesMessage`
  - `MapMessage` map values can be primitive types, strings, arrays or objects with corresponding accessors
    - Use `setInt, getString, setObject`, etc. to set map values

Entity Callbacks and Listeners

Chapter 14

Callback Events 1/2

- An entity's lifecycle has several phases
  - Each phase has its own annotation
- Entity creation
  - `@PrePersist` occurs when the entity is scheduled to be entered into the database
  - `@PostPersist` occurs immediately after the entity has been entered in the database
- Entity loading
  - `@PostLoad` occurs after an entity has been loaded by a `find()`, `getReference()`, JPA QL query, `Criteria` query, or `refresh()`

Callback Events 2/2

- Entity synchronized with database
  - `@PreUpdate` occurs when synchronization is scheduled
  - `@PostUpdate` occurs after the entity is synchronized with the database
- Removing an entity
  - `@PreRemove` occurs when the entity is scheduled to be removed
  - `@PostRemove` occurs after the entity is removed from the database

Entity Listeners

- Other classes can register themselves to receive entity callback events
  - class must have no-argument constructor
- Annotate methods with callback event annotation
- Method must return void and take one `Object` parameter
  - will be entity instance related to the event
- Entity being listened to must declare the listener(s) in `@EntityListeners` annotation
  - Takes array of classes as value argument

Security

Chapter 15

Example: SecureSchoolBean

- Annotation on bean
  - `@DeclareRoles({Roles.ADMIN, Roles.STUDENT, Roles.JANITOR})`
    - Declared `Roles` interface with `ADMIN="Administrator"`, etc
  - `@RolesAllowed({})` – no one by default
- Annotation on `isOpen()`
  - `@PermitAll`
- Annotation on `open(), close()`
  - `@RolesAllowed(Roles.ADMIN)`
- Annotation on `openServiceDoor()`
  - `@RolesAllowed({Roles.ADMIN, Roles.JANITOR})`

Transactions

Chapter 17

Transactions and EJBs

- Transactions are intended to increase application reliability
- A group of activities either all succeed or are all rolled back
  - debit checking account, credit savings account
- Desirable properties from database theory:
  - atomicity, consistency, isolation, durability (ACID)

ACID Transactions

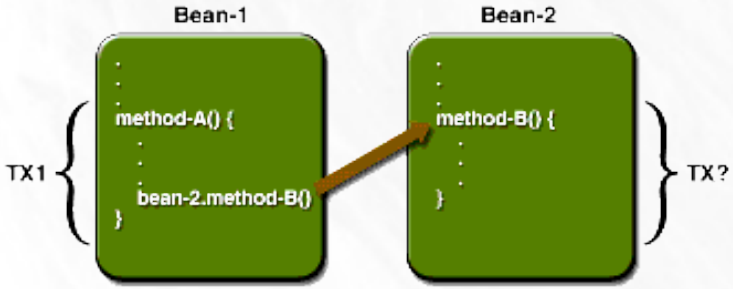
- Atomic
  - execute completely or not at all
- Consistent
  - data after operation must be in harmony with real-world represented by system
- Isolated
  - data accessed by transaction not affected by other parts of the system
- Durable
  - changes to data not lost if system crashes

Transaction Scope

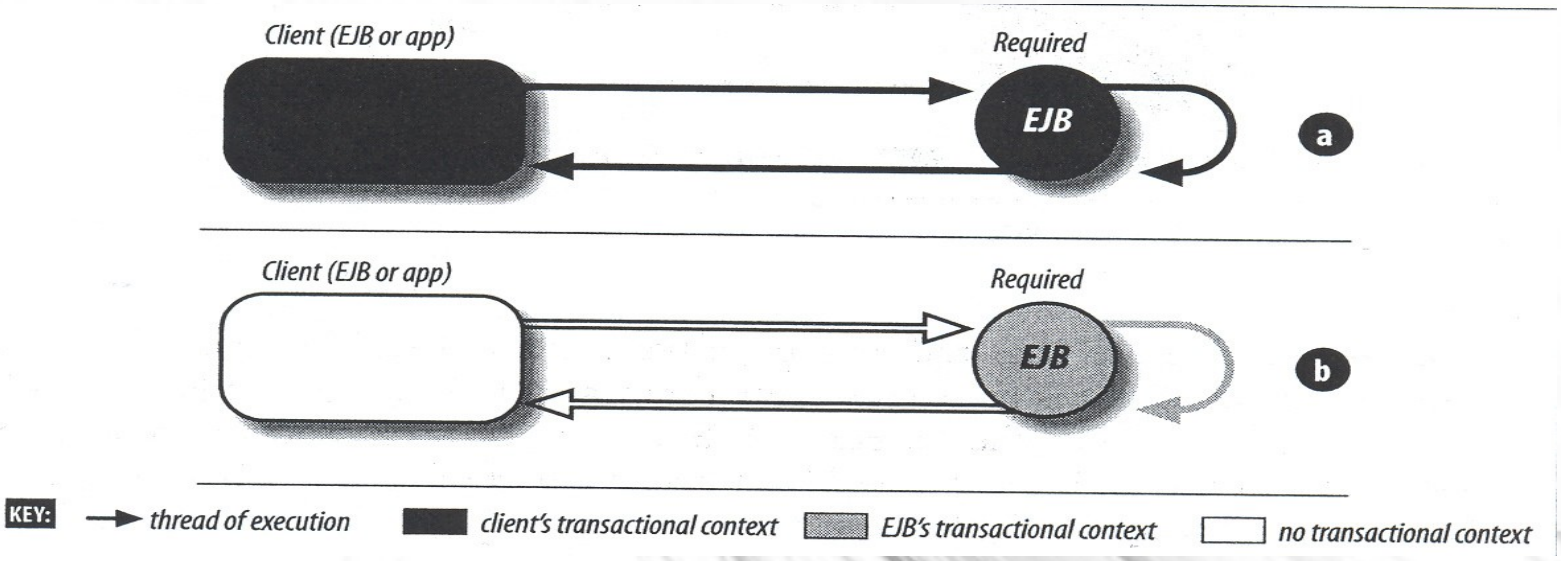
- Transaction scope means the EJBs that participate in a transaction
- In bookPassage() method of TravelAgent, all of the effected EJBs should be in the same transaction scope
- Transaction is propagated to EJB when its method is called and included in the scope of the calling EJB
- Transaction can end if exception is thrown as well as normal method completion

Transaction Attributes and Scope

- Suppose method-A calls method-B in a different bean



- Transaction attribute defines how method-B participates in method-A transaction
  - NotSupported, Supports, Required, RequiresNew, Mandatory, Never
  - declared in @TransactionAttribute or ejb-jar.xml

Transaction Attributes: Required

- If client is within transaction, the method executes in the client's transaction
- Client not associated with transaction, container starts a new transaction for this method
- Will work for most cases
- Makes a good default